

Bài 1: Tổng quan (Số tiết: 5 tiết)

1.1. Tổng quan lập trình hướng chức năng và lập trình hướng đối tượng

1.1.1. Lập trình tuyến tính

Lập trình tuyến tính có 2 đặc trưng cơ bản sau:

- Đơn giản: chương trình được tiến hành đơn giản theo lối tuần tự, không phức tạp.
- Đơn luồng: Chỉ có một luồng công việc duy nhất, các công việc được thực hiện tuần tự trong các luồng đó.

Ưu điểm: Chương trình đơn giản, dễ hiểu.

Nhược điểm: Không thể dùng lập trình tuyến tính để giải quyết các bài toán phức tạp.

Ngày nay, lập trình tuyến tính chỉ tồn tại trong phạm vi các module nhỏ nhất của các phương pháp lập trình khác.

1.1.2. Lập trình có cấu trúc

Cách tiếp cận theo hướng thủ tục dựa vào chức năng, nhiệm vụ là chính và Phân rã chức năng làm mịn dần theo cách từ trên xuống (Top/Down). Các đơn thể chức năng trao đổi với nhau bằng cách truyền tham số hay sử dụng dữ liệu chung.

- **Dựa vào chức năng nhiệm vụ:** Khi khảo sát, phân tích một hệ thống chúng ta thường tập trung vào các nhiệm vụ mà nó cần thực hiện. Chúng ta tập trung trước hết nghiên cứu các yêu cầu của bài toán để xác định các chức năng chính của hệ thống.

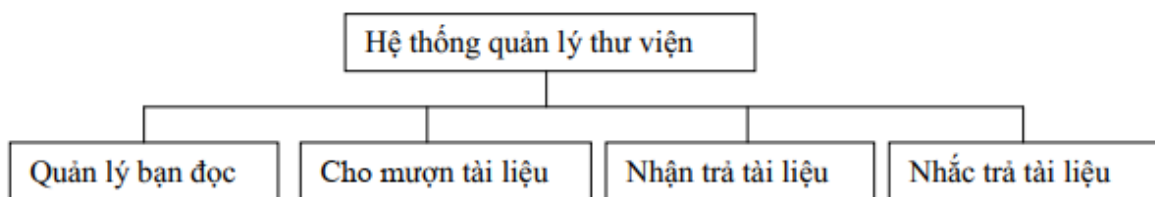
Ví dụ khi cần xây dựng “hệ thống quản lý thư viện” thì trước hết chúng ta thường đi nghiên cứu, khảo sát trao đổi và phỏng vấn xem những người thủ thư, bạn đọc cần phải thực hiện những công việc gì để phục vụ được bạn đọc và quản lý tốt được các tài liệu.

Qua nghiên cứu “hệ thống quản lý thư viện”, chúng ta xác định được các nhiệm vụ chính của hệ thống như: quản lý bạn đọc, cho mượn sách, nhận trả sách, thông báo nhắc trả sách, v.v

Hệ thống phần mềm được xem như là tập các chức năng, nhiệm vụ cần tổ chức thực thi.

- **Phân rã chức năng làm mịn từ trên xuống:** Khả năng của con người là có giới hạn khi khảo sát, nghiên cứu để hiểu và thực thi những gì mà hệ thống thực tế đòi hỏi.

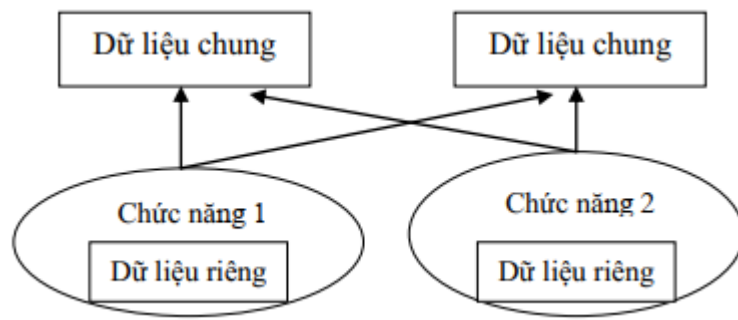
Để thống trị (quản lý được) độ phức tạp của những vấn đề phức tạp trong thực tế thường chúng ta phải sử dụng nguyên lý chia để trị (divide and conquer), nghĩa là phân tách nhỏ các chức năng chính thành các chức năng đơn giản hơn theo cách từ trên xuống. Quy trình này được lặp lại cho đến khi thu được những đơn thể chức năng tương đối đơn giản, hiểu được và thực hiện cài đặt chúng mà không làm tăng thêm độ phức tạp để liên kết chúng trong hệ thống. Độ phức tạp liên kết các thành phần chức năng của hệ thống thường là tỉ lệ nghịch với độ phức tạp của các đơn thể. Vì thế một vấn đề đặt ra là có cách nào để biết khi nào quá trình phân tách các đơn thể chức năng hay còn gọi là quá trình làm mịn dần này kết thúc. Thông thường thì quá trình thực hiện phân rã các chức năng của hệ thống phụ thuộc nhiều vào độ phức hợp của bài toán ứng dụng và vào trình độ của những người tham gia phát triển phần mềm. Một hệ thống được phân tích dựa trên các chức năng hoặc quá trình sẽ được chia thành các hệ thống con và tạo ra cấu trúc phân cấp các chức năng. Ví dụ, hệ thống quản lý thư viện có thể phân chia từ trên xuống như sau:



Hình 1.1: Sơ đồ chức năng của Hệ thống quản lý thư viện

Các chức năng của hầu hết các hệ thống thông tin quản lý đều có thể tổ chức thành sơ đồ chức năng theo cấu trúc phân cấp có thứ bậc.

- Các đơn thể chức năng trao đổi với nhau bằng cách truyền tham số hay sử dụng dữ liệu chung. Một hệ thống phần mềm bao giờ cũng phải được xem như là một thể thống nhất, do đó các đơn thể chức năng phải có quan hệ trao đổi thông tin, dữ liệu với nhau. Trong một chương trình gồm nhiều hàm (thực hiện nhiều chức năng khác nhau) muốn trao đổi dữ liệu được với nhau thì nhất thiết phải sử dụng dữ liệu chung hoặc liên kết với nhau bằng cách truyền tham biến. Mỗi đơn thể chức năng không những chỉ thao tác, xử lý trên những dữ liệu cục bộ (Local Variables) mà còn phải sử dụng các biến chung, thường đó là các biến toàn cục (Global Variable).



Hình 1.2: Mối quan hệ giữa các chức năng trong hệ thống

Nhận xét:

- Hệ thống được xây dựng dựa vào chức năng là chính mà trong thực tế thì chức năng, nhiệm vụ của hệ thống lại hay thay đổi. Để đảm bảo cho hệ thống thực hiện được công việc theo yêu cầu, nhất là những yêu cầu về mặt chức năng đó lại bị thay đổi là công việc phức tạp và rất tốn kém.

Ví dụ: giám đốc thư viện yêu cầu thay đổi cách quản lý bạn đọc hoặc hơn nữa, yêu cầu bổ sung chức năng theo dõi những tài liệu mới mà bạn đọc thường xuyên yêu cầu để đặt mua, v.v. Khi đó vấn đề bảo trì hệ thống phần mềm không phải là vấn đề dễ thực hiện. Nhiều khi có những yêu cầu thay đổi cơ bản mà việc sửa đổi không hiệu quả và vì thế đòi hỏi phải phân tích, thiết kế lại hệ thống thì hiệu quả hơn.

- Các bộ phận của hệ thống phải sử dụng biến toàn cục để trao đổi với nhau, do vậy khả năng thay đổi, mở rộng của chúng và của cả hệ thống là bị hạn chế. Như trên đã phân tích, những thay đổi liên quan đến các dữ liệu chung sẽ ảnh hưởng tới các bộ phận liên quan. Do đó, một thiết kế tốt phải rõ ràng, dễ hiểu và mọi sửa đổi chỉ có hiệu ứng cục bộ.
- Khả năng tái sử dụng (Reuse) bị hạn chế và không hỗ trợ cơ chế kế thừa (Inheritance). Để có độ thích nghi cao thì mỗi thành phần phải là tự chứa. Muốn là tự chứa hoàn toàn thì một thành phần không nên dùng các thành phần ngoại lai. Tuy nhiên, điều này lại mâu thuẫn với kinh nghiệm nói rằng các thành phần hiện có nên là dùng lại được. Vậy là cần có một sự cân bằng giữa tính ưu việt của sự dùng lại các thành phần (ở đây chủ yếu là các hàm) và sự mất mát tính thích ứng được của chúng. Các thành của hệ thống phải kết dính (Cohension) nhưng phải tương đối lỏng để dễ thích nghi. Một trong cơ chế chính hỗ trợ để dễ có được tính thích nghi là kế thừa thì cách tiếp cận hướng chức năng lại không hỗ trợ. Đó là cơ chế biểu diễn tính tương tự của các thực thể, đơn giản hoá định nghĩa những khái niệm tương tự từ những sự

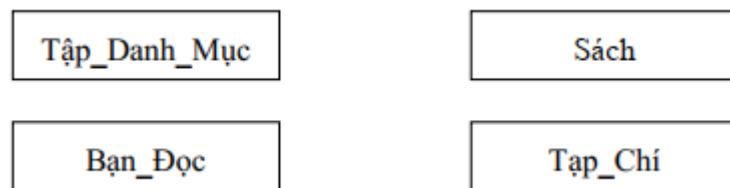
vật đã được định nghĩa trước trên cơ sở bổ sung hay thay đổi một số các đặc trưng hay tính chất của chúng. Cơ chế này giúp chúng ta thực hiện được nguyên lý tổng quát hoá và chi tiết hoá các thành phần của hệ thống phần mềm.

1.1.3. Lập trình hướng đối tượng

Dựa trên nền tảng là các đối tượng: Đặt trọng tâm vào dữ liệu, xem hệ thống là tập các thực thể và tập các đối tượng.

- Đặt trọng tâm vào dữ liệu (thực thể). Khi khảo sát, phân tích một hệ thống chúng ta tìm hiểu xem nó gồm những thực thể nào. Thực thể hay còn gọi là đối tượng, là những gì như người, vật, sự kiện, v.v. mà chúng ta đang quan tâm, hay cần phải xử lý. Ví dụ, khi xây dựng “Hệ thống quản lý thư viện” thì trước hết chúng ta tìm hiểu xem nó gồm những lớp đối tượng hoặc những khái niệm nào.

- Xem hệ thống như là tập các thực thể, các đối tượng. Để hiểu rõ về hệ thống, chúng ta phân tách hệ thống thành các đơn thể đơn giản hơn. Quá trình này được lặp lại cho đến khi thu được những đơn thể tương đối đơn giản, dễ hiểu và thực hiện cài đặt chúng mà không làm tăng thêm độ phức tạp khi liên kết chúng trong hệ thống. Xét “Hệ thống quản lý thư viện”, chúng ta có các lớp đối tượng sau:



Hình 1.3 Tập các lớp đối tượng của hệ thống

- Các lớp đối tượng trao đổi với nhau bằng các thông điệp (Message). Theo nghĩa thông thường thì lớp (Class) là nhóm một số người, vật có những đặc tính tương tự nhau hoặc có những hành vi ứng xử giống nhau. Trong mô hình đối tượng, khái niệm lớp là cấu trúc mô tả hợp nhất các thuộc tính (Attributes), hay dữ liệu thành phần (Data Member) thể hiện các đặc tính của mỗi đối tượng và các phương thức (Methods), hay hàm thành phần (Member Function) thao tác trên các dữ liệu riêng và là giao diện trao đổi với các đối tượng khác để xác định hành vi của chúng trong hệ thống. Khi có yêu cầu dữ liệu để thực hiện một nhiệm vụ nào đó, một đối tượng sẽ gửi một thông điệp (gọi một phương thức) cho đối tượng khác. Đối tượng nhận được thông điệp yêu cầu sẽ phải thực hiện một số công việc trên các dữ liệu mà nó sẵn có hoặc lại tiếp tục yêu cầu những đối tượng khác hỗ trợ để có những thông tin trả lời cho đối tượng yêu cầu. Với phương

thức xử lý như thế thì một chương trình hướng đối tượng thực sự có thể không cần sử dụng biến toàn cục nữa.

1.2. Các khái niệm cơ bản của lập trình hướng đối tượng

1.2.1. Đối tượng

- Đối tượng là thực thể của hệ thống được xác định thông qua định danh (tên gọi).

- Mỗi đối tượng bao gồm dữ liệu thành phần hay các thuộc tính mô tả các tính chất và phương thức thao tác trên dữ liệu để xác định hành vi của đối tượng.

Đối tượng = Thuộc tính (dữ liệu) + phương thức (hàm).

- Theo quan điểm của người lập trình đối tượng được xem như vùng nhớ được phân chia trong máy tính để lưu dữ liệu và tập các hàm tác động lên dữ liệu gắn với chúng.

Ví dụ: Quyển sách = thông tin về sách (khổ sách, độ dày, nội dung...) + Đọc (mở sách, lật trang, đọc 1 đoạn, đọc một trang, tìm mục lục...)

(đối tượng) = Thuộc tính + phương thức

- Ta có thể đọc sách dưới ánh đèn.

Đèn = Thông tin trạng thái (sáng, tắt) + bật, tắt đèn.

- Quyển sách chứa nội dung thông tin, nó cũng bao gồm các thông tin về trạng thái (sách đang mở, đóng) và có thể chứa các đối tượng nhỏ hơn (các trang sách). Phương thức của sách cho phép truy cập đến trang và phương thức của trang truy cập đến thông tin trong trang đó.

- Đối tượng có thể là một thực thể trực quan (người, sự vật) hay là một khái niệm (phòng ban, bộ phận, đăng ký...)

- Ta sẽ gọi tập hợp giá trị các thuộc tính của một đối tượng cụ thể tại một thời điểm nào đó là trạng thái (state) của nó.

- Một hành vi là cái mà đối tượng có thể thực hiện, và sẽ bằng cách nào đó tác động tới một hoặc nhiều thuộc tính của đối tượng (do đó ảnh hưởng đến trạng thái của đối tượng). *Xét một người có một thuộc tính tư thế - một trong các giá trị {đứng, ngồi, nằm, quỳ}* Một hành vi “đứng” có thể dẫn đến kết quả là người đó đứng dậy (nếu người đó đang không ở tư thế đứng)

- Trao đổi thông điệp: Các đối tượng liên lạc với nhau bằng các thông điệp (message). Một thông điệp là một yêu cầu một đối tượng thực hiện một hành vi.

Ví dụ: nếu ta muốn một người nào đó đứng dậy, ta có thể gửi một thông điệp đến cho người đó để đề nghị người đó đứng dậy. Đối với con người, phần lớn việc “truyền thông điệp” (messagepassing) của ta được thực hiện qua đường tiếng nói. Ta sẽ thấy rằng cơ chế “truyền thông điệp” đó xảy ra trong lập trình hướng đối tượng qua các lời gọi hàm.

- Trong mô hình hướng đối tượng: các đối tượng tương tác với nhau để thực hiện nhiệm vụ.

* Phân loại đối tượng:

Khả năng phân loại bẩm sinh của con người, khả năng trừu tượng hoá này cho phép ta đơn giản hoá và tổ chức cuộc sống của mình, và hiểu về một thế giới rất phức tạp. Có hai cách phân loại đối tượng mà ta hay dùng nhất: phân loại các đối tượng tương tự và phân loại các đối tượng theo thành phần.

+ Phân loại các đối tượng tương tự: nhóm các đối tượng có cùng các thuộc tính và hành vi lại với nhau. Ví dụ, trong loại “ô tô”, ta có thể đưa vào đó các đối tượng có các thuộc tính màu sắc, nhãn hiệu, kiểu, và các hành vi lái, dừng, rẽ.

Lưu ý: phân loại theo "có hay không một thuộc tính", không phân loại theo giá trị của thuộc tính. Ở đây, ta không lập nhóm các xe ô tô có giá trị “đỏ” cho thuộc tính màu và giá trị “Honda” cho thuộc tính nhãn hiệu đối tượng chỉ cần có các thuộc tính đó là đủ để thuộc nhóm.

+ Phân loại theo cấu tạo: Thuộc tính của một lớp nào đó có thể có giá trị là một đối tượng thuộc một lớp nào đó.

+ Phân loại đối tượng theo cấu tạo là một trong các kiểu *quan hệ* có thể tồn tại giữa các đối tượng. Kiểu quan hệ này thường được gọi là quan hệ chứa “has-a”.

* Mô hình hóa: Bước đầu tiên khi mô hình hoá một hệ thống gồm các lớp đối tượng, ta xác định xem:

- Có các lớp đối tượng nào?
- Chúng có các thuộc tính gì?
- Chúng có liên quan đến nhau như thế nào?
- Việc xác định hành vi của các lớp sẽ để sau

Ôn định tập thuộc tính cho mỗi lớp trước khi xác định các hành vi dùng để truy nhập các thuộc tính này.

Khi đã xác định được tất cả các lớp đối tượng cần thiết, các thuộc tính và quan hệ giữa chúng, bước tiếp theo là chỉ ra các hành vi của các lớp đối tượng đó.

- Có những hành vi nào?
- Các lớp đối tượng có thể cần những hành vi nào để có thể sử dụng các thuộc tính của mình?
- Các lớp đối tượng cần thực hiện những nhiệm vụ nào cho các đối tượng khác?

1.2.2. Lớp đối tượng

- Ta sẽ gọi một nhóm các đối tượng tương tự là một lớp đối tượng – object class. Một lớp là “một tập các đối tượng có cùng thuộc tính và hành vi”

- Mỗi đối tượng thuộc một lớp là một thể hiện (instance) của lớp đó. *Mỗi xe ô tô là một thể hiện của lớp ô tô.*

- Các thể hiện của một lớp phân biệt nhau bởi một số thuộc tính nhất định có giá trị duy nhất trong toàn lớp - định danh duy nhất “*định danh của sinh viên là mã sinh viên* – không có hai sinh viên nào có mã giống nhau đôi khi không dễ tìm được cách phân biệt giữa các thể hiện

- Lớp định nghĩa kiểu dữ liệu và phương thức dùng để truy cập dữ liệu của đối tượng.

Ví dụ: Lớp đơn xin việc = trường cần điền vào đơn + ghi/đọc đơn

(đối tượng) = (dữ liệu) + (phương thức)

Lớp này là mẫu đơn xin việc.

Những người đến xin việc họ phải được cung cấp một thực thể duy nhất của đơn, những thực thể tạo ra bằng cách lấy bản mẫu làm bản chính, sao chép tạo ra nhiều thực thể. Người xin việc phải điền các thông tin vào đơn bằng phương thức ghi đơn.

- Lớp được đặc tả bởi tên lớp: tập thuộc tính, tập các hàm.

Ví dụ: lớp sinh viên có 4 thuộc tính, 2 hàm (phương thức)

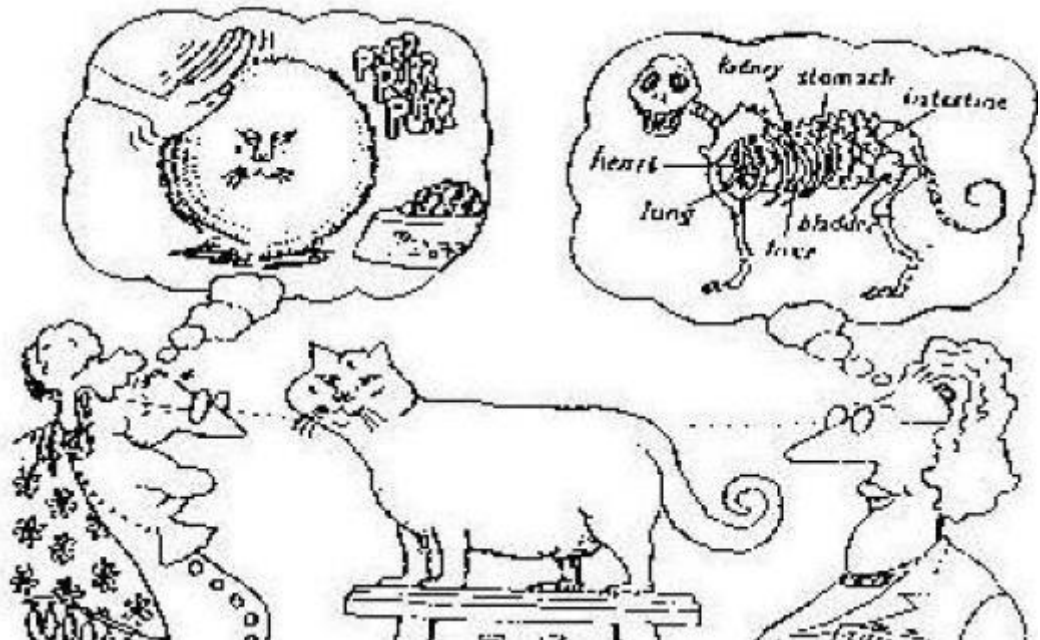
Đối tượng: sinh viên Nam thuộc lớp sinh viên nên nó mang đầy đủ thuộc tính và phương thức như lớp sinh viên.

<i>Nam</i>
<i>Hoten</i>
<i>tuoi</i>
<i>quequan</i>
<i>lop</i>
<i>getTen</i>
<i>getTuoi</i>
...

<i>SinhVien</i>
<i>Hoten</i>
<i>tuoi</i>
<i>quequan</i>
<i>lop</i>
<i>getTen</i>
<i>getTuo...</i>

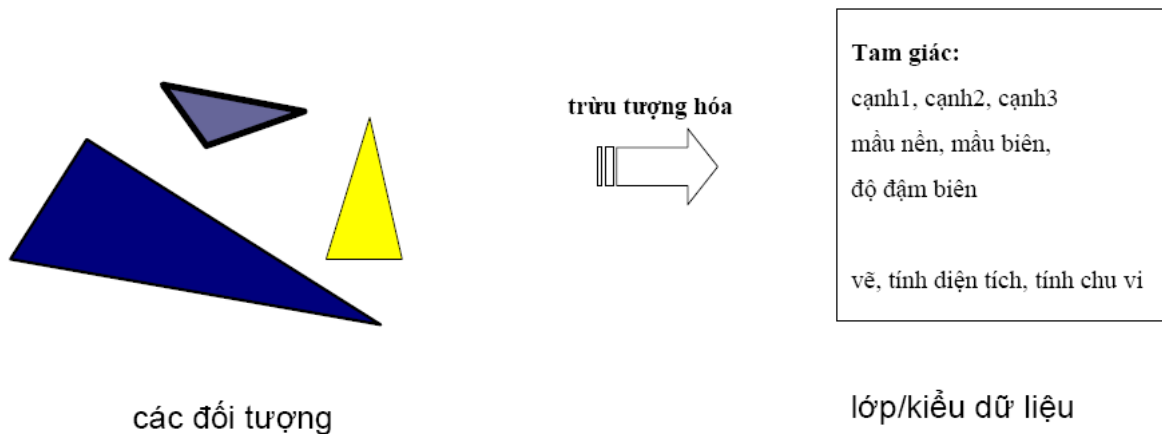
1.2.3. Trừu tượng hóa

Trừu tượng hóa là cách biểu diễn đặc tính chính và bỏ qua những chi tiết.



Hình 1.4 Trừu tượng hoá

Trừu tượng hóa là cách nhìn đơn giản hóa về một đối tượng mà trong đó chỉ bao gồm các đặc điểm được quan tâm và bỏ qua những chi tiết không cần thiết.



Hình 1.5 Sử dụng trừu tượng hoá để xây dựng lớp

- Trừu tượng hóa là sự mở rộng các khái niệm kiểu dữ liệu và cho phép định nghĩa những phép toán trừu tượng trên các kiểu dữ liệu trừu tượng.
- Để xác định lớp ta phải sử dụng khái niệm trừu tượng hóa.

1.2.4. Bao bọc và che dấu thông tin

- Đóng gói: Nhóm những gì có liên quan với nhau vào làm một, để sau này có thể dùng một cái tên để gọi đến: Các hàm/ thủ tục đóng gói các câu lệnh. Các đối tượng đóng gói dữ liệu của chúng và các thủ tục có liên quan.

Chi tiết về các thành viên của một đối tượng cần được đóng gói bên trong đối tượng, để cách duy nhất truy nhập đến chúng là qua các hành vi của đối tượng. Tương tự, chi tiết về cách thực hiện các hành vi của một đối tượng cũng nên được đóng gói để bên ngoài không biết chi tiết.

Tại sao đóng gói đối tượng? Đóng gói bắt buộc thế giới thực: nếu tôi muốn anh đứng dậy (thay đổi thuộc tính “vị trí”) khả năng lớn là tôi sẽ đề nghị anh đứng dậy thay vì đến kéo anh dậy. “ điều này làm đơn giản hoá cuộc sống, ta thường không quan tâm đến chi tiết. Trong lập trình hướng đối tượng, lợi ích của việc đóng gói: “ khi che dấu chi tiết cài đặt, người dùng bên ngoài không phải bận tâm đến chuyện cái gì được làm như thế nào. Điều này đem lại lợi ích đáng kể trong việc bảo trì phần mềm – do người dùng không bao giờ biết chi tiết bên trong đối tượng, ta có thể thay đổi các chi tiết đó mà họ không cần biết (miễn là giao diện với bên ngoài không đổi)

- Xác định các vùng: riêng (private) công khai (public) hay bảo vệ (protected) nhằm điều khiển, hay hạn chế những truy nhập tùy tiện của những đối tượng khác.

- Việc đóng gói dữ liệu và hàm vào cùng một đơn vị cấu trúc (lớp) được xem là nguyên tắc bao bọc thông tin.

- Nguyên tắc bao bọc dữ liệu để cấm sự truy nhập trực tiếp gọi là sự che dấu thông tin.

Che dấu thông tin: đóng gói để che một số thông tin và chi tiết cài đặt nội bộ để bên ngoài không nhìn thấy. mục tiêu là để khách hàng của ta (thường là các lập trình viên khác) coi các đối tượng của ta là các hộp đen.

1.2.5. Kế thừa và mở rộng

- Nguyên lý kế thừa cho phép các đối tượng của lớp này được quyền sử dụng lại một số tính chất (thuộc tính và phương thức) của lớp khác.

- A: lớp cha; B: lớp con. Lớp B sẽ kế thừa các thuộc tính và các hàm có khai báo public hoặc protected của lớp A.

- Trong lớp B có thể bổ sung thêm một số tính chất, phương thức truy cập để thu hẹp phạm vi xác định đối tượng trong lớp mới.

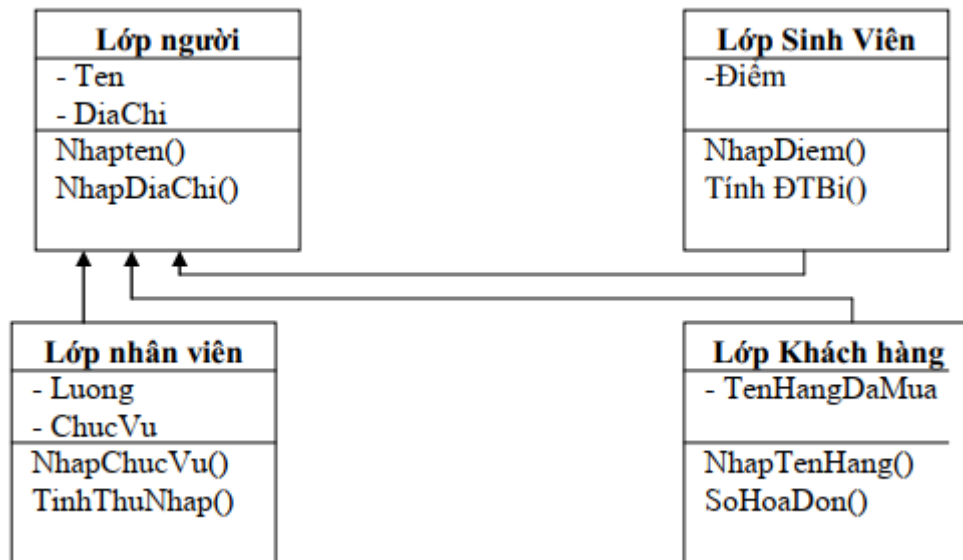
- Những thuộc tính hay hàm được khai báo private thì không kế thừa lại được.

Đây là sự kế thừa đơn, trong java không có sự kế thừa bội. Để tận dụng lợi ích của kế thừa bội java xây dựng khái niệm interface (giao diện).

Ví dụ: Trong bài toán có 3 lớp:

- * Lớp sinh viên: - Thuộc tính: tên, địa chỉ, điểm và các hàm: `nhapTen()`, `nhapDiaChi()`, `nhapDiem()`.

- * Lớp khách hàng: - Thuộc tính: tên, địa chỉ, tên hàng đã mua, và các hàm: `nhapTen()`, `nhapDiaChi()`, `nhapTenHang()`, `nhapHoaDon()`



Hình 1.6 Ví dụ kế thừa

1.2.6. Đa xạ và nạp chồng

➤ Đa hình:

- Là kỹ thuật sử dụng để mô tả khả năng gửi một thông điệp chung tới nhiều đối tượng mà mỗi đối tượng lại có cách xử lý riêng theo ngữ cảnh của mình.

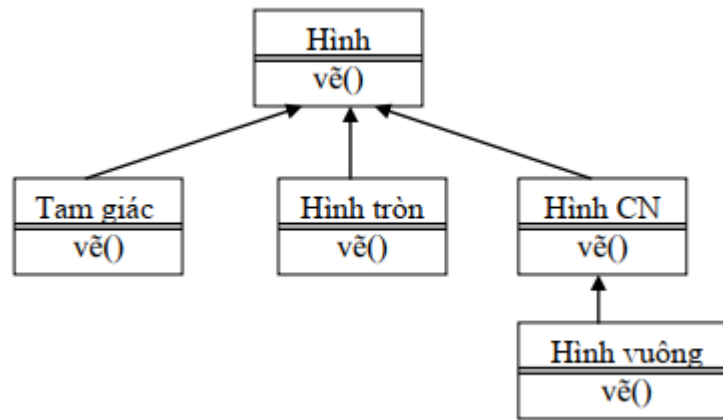
- Khả năng của đối tượng có nhiều phương thức cùng tên nhưng nội dung thực hiện khác nhau. Một thông điệp (lời gọi hàm) được hiểu theo các cách khác nhau tùy theo danh sách tham số của thông điệp. Ví dụ: nhận được cùng một thông điệp “nhảy”, một con kangaroo và một con cóc nhảy theo hai kiểu khác nhau: chúng cùng có hành vi “nhảy” nhưng các hành vi này có nội dung khác nhau.



chú ý rằng kangaroo và cóc thuộc hai nhánh trong cây phả hệ động vật

Hình 1.7 Đa hình

Ví dụ: Hàm vẽ là hàm đa trị. Nó được xác định tùy ngữ cảnh khi sử dụng. Khi sử dụng làm vẽ với đối tượng tam giác thì thực hiện việc vẽ tam giác. Khi sử dụng hàm vẽ với đối tượng hình chữ nhật thì thực hiện việc vẽ hình chữ nhật...



Hình 1.8 Ví dụ đa hình

➤ Nạp chồng:

- Nạp chồng là trường hợp đặc biệt của đa trị. Dùng một tên hàm cho nhiều định nghĩa hàm, nhưng khác nhau ở kiểu của tham số hoặc số lượng tham số thì khi đó nội dung thực trong mỗi hàm là khác nhau.

Ví dụ: hàm cộng()

int cong(int, int)

float (float, float)

string cong (string, string)

int cong(int a, int b, int c)

int cong(int a[])// danh sách đầu vào là một mảng

1.2.7. Liên kết động

- Liên kết tĩnh: lời gọi hàm (phương thức) được quyết định khi biên dịch, do đó chỉ có một phiên bản của chương trình con được thực hiện. Ưu điểm về tốc độ

- Liên kết động: lời gọi hàm được quyết định khi thực hiện, phiên bản của phương thức phù hợp với đối tượng được gọi. Java mặc định sử dụng liên kết động.

Ví dụ: khi chương trình gặp hàm vẽ cho đối tượng hình tròn thì hệ thống sẽ liên kết tới hàm vẽ được định nghĩa trong lớp hình tròn.

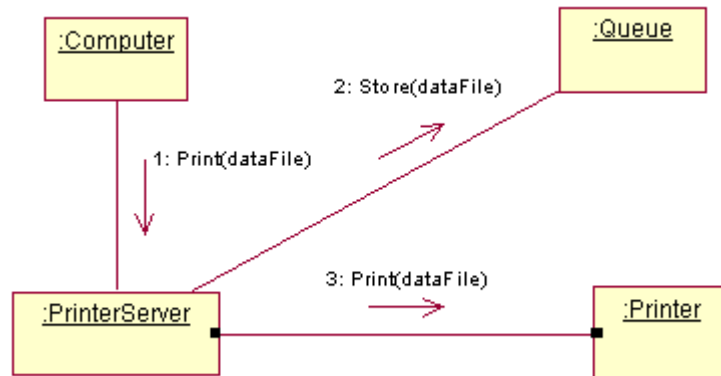
1.2.8. Truyền thông điệp

Chương trình hướng đối tượng bao gồm tập các đối tượng và mối quan hệ giữa các đối tượng đó với nhau. Lập trình trong ngôn ngữ hướng đối tượng bao gồm các bước sau:

1. Tạo ra các lớp đối tượng và mô tả chúng bằng các thuộc tính và hành vi của chúng
2. Tạo ra các đối tượng theo định nghĩa của các lớp.
3. Xác định sự trao đổi thông tin giữa các đối tượng trong hệ thống.

Truyền thông điệp cho một đối tượng chính là để yêu cầu thực hiện một công việc cụ thể nào đó, nghĩa là sử dụng những hàm tương ứng để xử lý dữ liệu đã được khai báo trong lớp đối tượng đó.

Ví dụ, khi hệ thống máy tính muốn in một tệp *dataFile* thì máy tính hiện thời (:Computer) gửi đến cho đối tượng :PrinterServer một yêu cầu *Print(dataFile)*.



Hình 1.9 Truyền thông điệp giữa các đối tượng

Mỗi đối tượng chỉ tồn tại trong thời gian nhất định. Đối tượng tạo ra khi nó được báo và sẽ được loại bỏ khi chương trình ra khỏi miền xác định của đối tượng đó. Sự trao đổi thông điệp chỉ thực hiện được trong thời gian tồn tại của đối tượng.

1.3. Các ngôn ngữ lập trình hướng đối tượng

Các ngôn ngữ lập trình thời kỳ đầu như Basic, Fortran... không có cấu trúc và cho phép viết những đoạn mã rối rắm (spaghetti code). Lập trình viên sử dụng các lệnh goto” và “gosub” để nhảy đến mọi nơi trong chương trình. Đoạn trình trên khó theo dõi, khó hiểu, dễ gây lỗi, khó sửa đổi.

Kiểu lập trình rối rắm trên dẫn tới phong cách lập trình mới: lập trình cấu trúc, với các ngôn ngữ Algol, Pascal, C... Đặc điểm của lập trình cấu trúc hay lập trình thủ tục (Procedural Programming- PP) là:

- Sử dụng các cấu trúc vòng lặp: for, while, repeat, do-while
- Chương trình là một chuỗi các hàm/ thủ tục
- Mã chương trình tập trung thể hiện thuật toán: làm như thế nào

Hạn chế của lập trình thủ tục:

- Dữ liệu và phần xử lý tách biệt
- Dữ liệu thụ động, xử lý chủ động
- Không đảm bảo được tính nhất quán và các ràng buộc của dữ liệu
- Khó cảm mã ứng dụng sửa dữ liệu của thư viện

- Khó bảo trì code
- Phần xử lý có thể nằm rải rác và phải hiểu rõ cấu trúc dữ liệu

Lập trình hướng đối tượng: *Lập trình hướng đối tượng cho phép khắc phục các hạn chế nói trên*

Các ngôn ngữ lập trình hướng đối tượng không mới: " Simula (1967) là ngôn ngữ đầu tiên, có lớp, thừa kế, liên kết động (hay còn gọi là hàm ảo). *Nhưng các ngôn ngữ hướng đối tượng chậm hơn các ngôn ngữ thời kỳ đầu nên chúng chỉ được dùng rộng rãi khi máy tính bắt đầu chạy nhanh (khoảng thời gian chiếc máy Pentium đầu tiên ra đời).*

Dựa vào khả năng đáp ứng các khái niệm về hướng đối tượng, ta chia ra làm hai loại:

* Ngôn ngữ lập trình dựa trên đối tượng (object-based):

Lập trình dựa trên đối tượng là kiểu lập trình hỗ trợ chính cho việc bao bọc, che giấu thông tin và định danh các đối tượng. Lập trình dựa trên đối tượng có những đặc tính sau:

- Bao bọc dữ liệu,
- Cơ chế che giấu và hạn chế truy nhập dữ liệu,
- Tự động tạo lập và huỷ bỏ các đối tượng,
- Tính đa xạ.

Ngôn ngữ hỗ trợ cho kiểu lập trình trên được gọi là ngôn ngữ lập trình dựa trên đối tượng. Ngôn ngữ trong lớp này không hỗ trợ cho việc thực hiện kế thừa và liên kết động. Ada là ngôn ngữ lập trình dựa trên đối tượng.

* Ngôn ngữ lập trình hướng đối tượng (object-oriented)

Lập trình hướng đối tượng là kiểu lập trình dựa trên đối tượng và bổ sung thêm nhiều cấu trúc để cài đặt những quan hệ về kế thừa và liên kết động. Vì vậy đặc tính của LTHĐT có thể viết một cách ngắn gọn như sau:

Các đặc tính dựa trên đối tượng + kế thừa + liên kết động.

Ngôn ngữ hỗ trợ cho những đặc tính trên được gọi là ngôn ngữ LTHĐT, ví dụ như Java, C++, Smalltalk, Object Pascal hay Eiffel, v.v... *Mức độ hướng đối tượng của các ngôn ngữ không giống nhau: Eiffel (tuyệt đối), Java (rất cao), C++ (nửa nọ nửa kia)*

Các bài tập thực hành mức độ cơ bản

Bài 1

1. Cài đặt JDK và cài đặt NetBean (hoặc eclipse), làm quen với các chức năng của NetBean IDE (hoặc Eclipse IDE), phân tích ưu nhược điểm của hai môi trường phát triển này.
2. Thực hiện cài đặt lại các bài giải mẫu
3. Làm quen, hiểu với một số kiểu lỗi cú pháp đơn giản và cách khắc phục

Bài 2 Viết chương trình:

1. Khai báo (import) lớp Scanner trong gói java.util
2. Sử dụng phương thức *nextLine()* của lớp Scanner để nhập vào một xâu và in xâu vừa nhập ra màn hình.

Bài 3 Viết chương trình:

1. Sử dụng lớp Scanner trong gói java.util

Sử dụng phương thức *nextInt()* để nhập vào ba số a, b, c tương ứng với độ dài của ba cạnh của một tam giác. in ra thông tin chu vi ($p=a+b+c$) của tam giác tương ứng.

+ Bài giải mẫu (có hướng dẫn cách giải)

Bài 1: HelloWorld

HelloWorld.java (tên file)

```
class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Bài 2 : Tính tổng của hai số được nhập từ bàn phím

CTTinhTong.java (tên file)

```
import java.util.Scanner;  
public class CTTinhTong {  
    public static void main(String args[]) {  
        Scanner w = new Scanner(System.in);  
        int a = 0, b = 0;  
        System.out.println("Nhap so a=");  
        a = w.nextInt();  
        System.out.println("Nhap so b=");  
        b = w.nextInt();  
        System.out.println("tong a+b=" + (a + b) + "hieu a-b=" + (a  
- b));  
    }  
}
```